

Práctica de Repaso

Semáforos

1. a. En una estación de trenes, asisten P personas que deben realizar una carga de su tarjeta SUBE en la terminal disponible. La terminal es utilizada en forma exclusiva por cada persona de acuerdo con el orden de llegada. Implemente una solución utilizando únicamente procesos Persona.

Versión 1

```
bool usando = false;
sem mutexCola = 1;
cola<int> cola_ids = new cola<int>();
sem[] sem_priv = ([P] 0);

process persona[Id: 1..P] {
    if (usando) // Quizás esto no garantiza realmente que pase el primero que ll
    egó
        P(mutexCola);
        cola_ids.push(Id);
        V(mutexCola);
        P(sem_priv[Id]);
        usando = true;
        UsarTerminal();
        if (!empty(cola_ids))
            P(mutexCola);
            int id = cola_ids.pop();
            V(mutexCola);
            V(sem_priv[Id]);
        else
```

```
    usando = false;
}
```

[Corregido]

1. b. Resuelva el mismo problema anterior pero ahora considerando que hay T terminales disponibles. Las personas realizan una única fila y la carga la realizan en la primera terminal que se libera. Recuerde que sólo debe emplear procesos Persona

```
sem mutexCola = 1;
cola<int> cola_ids = new cola<int>();
sem[] sem_priv = ([P] 0);

process persona[Id: 1..P] {
    P(mutexCola);
    cola_ids.push(Id); // Formar fila
    V(mutexCola)
    P(ocupados); // Si hay máquinas desocupadas...
    P(mutexCola);
    int aux_id = cola_ids.pop();
    V(mutexCola)
    if (aux_id != Id) // Si no soy la persona que saqué...
        P(sem_priv[Id]) // Dormirme y que me despierten cuando me toque el tur
no
    P(mutexCola_terminales);
    int id_ter = cola_terminales.pop(); // Sacar una terminal libre
    V(mutexCola_terminales);
    UsarTerminal(id_ter);
    P(mutexCola_terminales);
    cola_terminales.push(id_ter); // Meterla de nuevo para que alguien la use
    V(mutexCola_terminales);
    V(ocupados);
}
```

Versión 2 - Correcta

```
sem mutexCola = 1;
cola<int> cola_ids = new cola<int>();
sem[] sem_priv = ([P] 0);

process persona [Id: 1..P] {
    P(mutex);
    if (terminales_libres == 0)
        cola_ids.push(Id);
    V(mutex);
    P(sem_priv[Id]);
    terminales_libres--;
    V(mutex);

    P(mutex_terminales);
    id_ter = cola_terminales.pop();
    V(mutex_terminales);
    UsarTerminal(id_ter);
    P(mutex_terminales);
    cola_terminales.push(id_ter);
    V(mutex_terminales);

    P(mutex);
    terminales_libres++;
    if (!empty(cola_ids))
        id_persona = cola_ids.pop();
        V(sem_priv[id_persona]);
    V(mutex);
}
```

2. Implemente una solución para el siguiente problema. Un sistema debe validar un conjunto de 10000 transacciones que se encuentran disponibles en una estructura de datos. Para ello, el sistema dispone de 7 workers, los cuales trabajan

colaborativamente validando de a 1 transacción por vez cada uno. Cada validación puede tomar un tiempo diferente y para realizarla los workers disponen de la función *Validar(t)*, la cual retorna como resultado un número entero entre 0 al 9. Al finalizar el procesamiento, el último worker en terminar debe informar la cantidad de transacciones por cada resultado de la función de validación.

```
cola<Transaccion> cola_transacciones = // Supongamos que hay 10.000 transacciones aquí
sem mutexCola = 1;
sem mutex_vec_cant = 1;
sem mutex_termino = 1;
int[] cant_transacciones_tipo = ([9] 0);
int termino = 0;

process Worker[1..7] {
    int[] cant_transac_tipo_local = ([9] 0);

    while (True) { // Mientras hayan transacciones que procesar...
        P(mutexCola);
        transaccion transac = cola_transacciones.pop();
        V(mutexCola);
        int num_resul = Validar(transac);
        // Esto va a demorar un tiempo indeterminado
        cant_transac_tipo_local[num_resul]++;
    }
    P(mutex_vec_cant)
    for i:=0 to 9 do
        cant_transacciones_tipo[i] += cant_transac_tipo_local[i];
    V(mutex_vec_cant);
    P(mutex_termino) // ¿Quizás este semáforo debería ir adentro del de arriba?
    termino++;
    if (termino == 7)
        // Imprimir resultados
```

```
V(mutex_termino);  
}
```

INCORRECTO

3. Implemente una solución para el siguiente problema. Se debe simular el uso de una máquina expendedora de gaseosas con capacidad para 100 latas por parte de U usuarios. Además, existe un repositor encargado de reponer las latas de la máquina. Los usuarios usan la máquina según el ordende llegada. Cuando les toca usarla, sacan una lata y luego se retiran. En el caso de que la máquina se quede sin latas, entonces le debe avisar al repositor para que cargue nuevamente la máquina en forma completa. Luego de la recarga, saca una botella y se retira

```
// PASSING THE BATON  
cola<Lata> cola_maquina = new Cola<Lata>();  
cola<Int> cola_ids = new Cola<Int>();  
sem mutex_maq = 1;  
sem[] espera_id = ([U] 0);  
sem espera_reportes = 0;  
sem mutex = 1;  
bool usando = false;  
  
process usuario [Id: 1..U] {  
    P(mutex);  
    if (usando)  
        cola_ids.push(Id);  
        V(mutex);  
        P(espera_id[Id]);  
    else  
        usando = true;  
        V(mutex);  
  
    // Ya acá la atomicidad está garantizada, so  
    // Simular cruce
```

```

    if (empty(cola_maquina))
        V(espera_repositor) // Despertar repositor
        P(espera_usuario) // Quedarse esperando
    lata aux_lata = cola_maquina.pop();

    P(mutex);
    if (!empty(cola_ids))
        int aux_id = cola_ids.pop();
        V(mutex);
        V(espera_id[Id]);
    else
        V(mutex);
        usando = false;
    }

process repositor {
    while (True) { // Mientras siga habiendo gente...
        P(espera_repositor);
        lata[] aux_latas = cargarLata(20);
        cola_maquina.push(aux_latas);
        V(espera_usuario);
    }
}

```

1. Resolver el siguiente problema. En una elección estudiantil, se utiliza una máquina para voto electrónico. Existen N Personas que votan y una Autoridad de Mesa que les da acceso a la máquina de acuerdo con el orden de llegada, aunque ancianos y embarazadas tienen prioridad sobre el resto. La máquina de voto sólo puede ser usada por una persona a la vez. Nota: la función Votar() permite usar la máquina.

```

process persona [Id: 0..N-1] {
    int edad = ...;

```

```

bool embarazado = ...;

MaquinaVotar.pasar(id, edad, embarazado);
Votar();
MaquinaVotar.salir();
}

process autoridad {
    for i:=1 to N do
        MaquinaVotar.darAcceso();
    }

monitor MaquinaVotar {
    colaPrioridad<int, bool> cola_pri;
    cond espera_persona[id] = ([N]);
    cond espera_autoridad;
    cond espera_autoridad_2;

    procedure pasar(id: in int; edad: in int; embarazado: in bool) {
        cola_pri.push(id, edad, embarazado);
        signal(espera_autoridad);
        wait(espera_persona[id]);
    }

    procedure salir() {
        signal(espera_autoridad_2);
    }

    procedure dar_acceso() {
        if (empty(cola_pri))
            wait(espera_autoridad);
        int id = colaPrioridad.pop();
        signal(espera_persona[id]);
        wait(espera_autoridad_2);
    }
}

```

```
}  
}
```

[Corregido]

Comentarios que me hicieron en este ejercicio:

- Para plantear el tema de la cola de prioridad, podría mejor tener dos colas, una cola de prioridad solo para edad, y luego otra cola en la que solo se vayan encolando las personas que estén embarazadas. Me dijo esto porque dijo que era mucho mas fácil para entender, y que si en el parcial hacía la forma que hice en esta solución (plantear una cola que ordenaba tanto por edad como por embarazada), que tendría que detallarla o especificarla bien aparte, en un comentario, como funcionaría esta (sin mostrar código obviamente). Pero en principio no está mal, solo tendría que preguntarlo en el examen.

2. Resolver el siguiente problema. En una empresa trabajan 20 vendedores ambulantes que forman 5 equipos de 4 personas cada uno (cada vendedor conoce previamente a qué equipo pertenece). Cada equipo se encarga de vender un producto diferente. Las personas de un equipo se deben juntar antes de comenzar a trabajar. Luego cada integrante del equipo trabaja independientemente del resto vendiendo ejemplares del producto correspondiente. Al terminar cada integrante del grupo debe conocer la cantidad de ejemplares vendidos por el grupo. Nota: maximizar la concurrencia

```
process vendedor [1..20] {  
    int num_equipo;  
    BarreraEquipo[num_equipo].llegar();  
  
    int cont_ventas;  
    while (True) { // Mientras dure la jornada...  
        // Realizar una venta  
        cont_ventas++;  
    }  
}
```

```

    int recibir_puntos;
    VentaEquipo[num_equipo].sumar(cont_ventas, recibir_puntos);
}

monitor BarreraEquipo [1..5] {
    int cont_barrera = 0;
    cond espera;

    procedure Llegar() {
        if (cont_barrera < 5)
            wait(espera);
        else
            signal_all(espera);
    }
}

monitor VentaEquipo [1..5] {
    int cant_ventas;
    int cont_barrera = 0;
    cond espera;

    // "Cada integrante del grupo debe conocer la cantidad de ejemplares vendidos por el grupo"
    procedure sumar(ventas: in int; puntos: out int) {
        cont_barrera++;
        cant_ventas += ventas;

        if (cont_barrera < 5)
            wait(espera);
        else
            signal_all(espera);

        puntos = cant_ventas;
    }
}

```

[Corregido]

3. Resolver el siguiente problema. En una montaña hay 30 escaladores que en una parte de la subida deben utilizar un único paso de a uno a la vez y de acuerdo con el orden de llegada al mismo. Nota: sólo se pueden utilizar procesos que representen a los escaladores; cada escalador usa sólo una vez el paso.

```
// Ejemplo típico de Passing the Condition
process escalador [1..30] {
    Paso.usar();
    // Realizar paso
    Paso.salir();
}

monitor paso {
    cond espera;
    int esperando = 0;
    bool usando = false;

    procedure usar() {
        if (usando)
            esperando++;
        wait(espera);
        usando = true;
    }

    procedure salir() {
        if (esperando > 0)
            signal(espera);
        esperando--;
        else
            usando = false;
    }
}
```

[Corregido]